Stock Price Forecasting with XGBoost

In today's fast-moving stock market, retail investors and individual traders often struggle to make sense of endless price fluctuations and market noise. While professional traders rely on complex algorithms and data analysis, everyday investors rarely have access to similar predictive tools. This project addresses this gap by applying machine learning to forecast next-day performance of individual S&P 500 stocks from historical price data. The model uses XGBoost, a gradient boosting algorithm known for handling complex, nonlinear relationships while remaining robust against overfitting. It was chosen due to its demonstrated effectiveness in time-series forecasting contexts [1][2][3].

Traditional technical analysis, which uses indicators such as moving averages, RSI, or MACD to identify trends and momentum, has long divided investors. Some believe these indicators reveal meaningful market patterns, while others argue they merely reflect randomness in price movements. A key research goal of this project is to test whether incorporating common technical indicators alongside raw price and volume data can improve the predictive accuracy of a machine learning model.

By turning raw market data into actionable insights, the model helps identify potential opportunities and risks before the market opens. The goal is not to guarantee profits, but to enhance decision-making with data-driven predictions, giving retail investors an edge in understanding short-term stock movements.

Collecting Data

A reliable source of historical stock data is essential for any predictive modeling task. Initially, several APIs such as Yahoo Finance, Finnhub, and Tiingo were considered for collecting daily stock data. However, for this project, the yfinance library in Python proved to be the most convenient and robust solution. It provides quick access to daily Open, High, Low, Close, and Volume (OHLCV) data for all S&P 500 companies without the need for API keys or complex authentication.

Inspecting Data

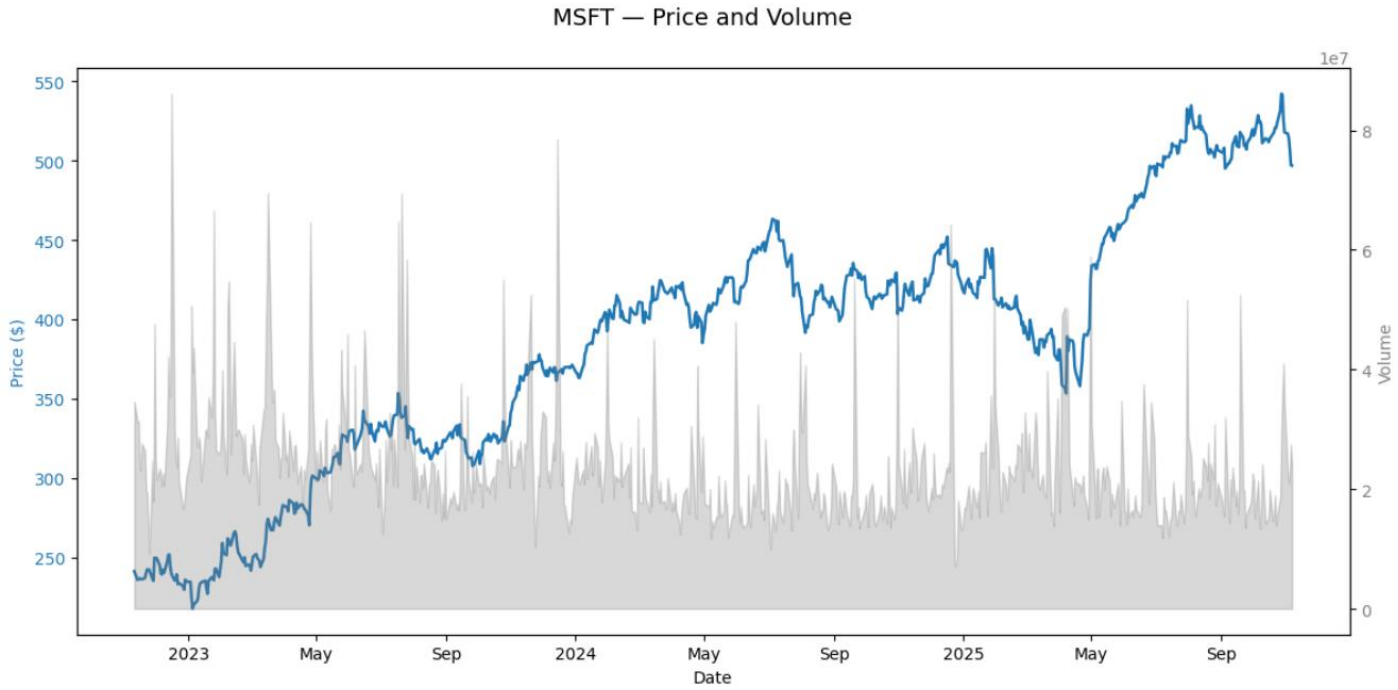
Once the data is collected, the next step is to carefully inspect and clean it to ensure the model receives consistent and reliable input. This involves checking for missing values, understanding the range and distribution of prices and trading volumes, and identifying any potential outliers that could distort predictions.

As an example, Microsoft (MSFT) stock data is inspected over the past three years. The dataset contains 750 trading days, and has no missing values in any of the key columns (Open, High, Low, Close, Volume). This makes it well-suited for modeling.

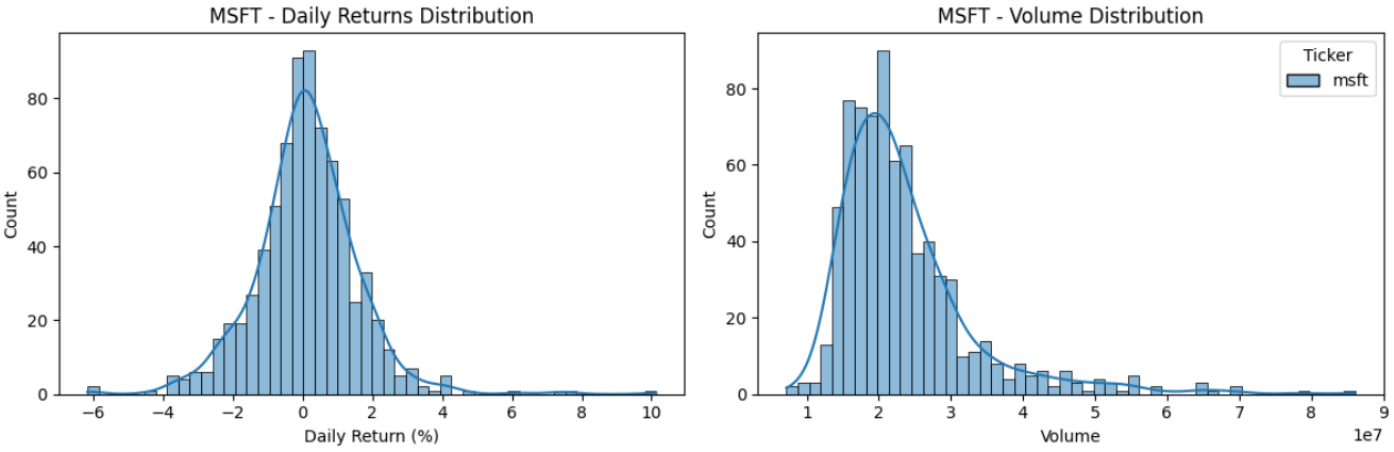
Descriptive statistics provide a clear picture of the data:

Price	Ticker	Count	Mean	Std	Min	Max
Close	MSFT	750	384.8	79.5	217.5	542.1
High	MSFT	750	388.1	79.9	220.9	554.5
Low	MSFT	750	381.3	79.3	214.6	540.8
Open	MSFT	750	384.8	79.8	218.2	554.3
Volume	MSFT	750	23,634,120	9,588,958	7,164,500	86,102,000

To better understand Microsoft's stock, closing prices (blue) and trading volumes (gray) are plotted. This shows price trends and highlights days with unusually high or low trading activity, helping spot patterns and anomalies before modeling.



Daily return and volume distributions are also examined. The histogram shows that most daily returns are within ±1%, with occasional larger moves, while the volume distribution highlights periods of higher trading activity. This offers an understanding of daily price variations and volatility patterns ahead of modeling.



Feature Engineering

Before feeding the stock data into a predictive model, it is important to transform raw OHLCV (Open, High, Low, Close, Volume) data into features that capture relevant patterns for forecasting the next day's price. The daily stock data is transformed into a format suitable for machine learning, where information from previous days is used to help predict the next day's closing price. Two approaches are explored: first, using only lagged OHLCV values as features, and second, augmenting the data with technical indicators commonly used in trading analysis. This allows us to compare whether adding technical indicators improves the model's predictive accuracy.

Technical indicators summarize price behavior and trends over time. Features such as moving averages (MA), relative strength index (RSI), momentum, volatility, MACD, and Bollinger Bands are included in the model. These indicators may capture patterns like short-term trends, overbought or oversold conditions, and price volatility, potentially providing the model with more predictive power than raw OHLCV alone.

Model Training and Evaluation

Once the features are prepared, the next step is to train and evaluate the predictive model. The model is repeatedly trained on a rolling window of past data and tested on the following day's price, mimicking real trading conditions. This walk-forward validation ensures that predictions are always made using only historical information, closely reflecting how a trader would use the model in practice.

Different configurations are tested, including how many past days to use as input and the size of the training window. The optimal values for these parameters are not universal, they differ between stocks due to variations in volatility, liquidity, and trading patterns. For example, a highly volatile stock may benefit from a shorter lag to capture rapid changes, while a more stable stock may require a longer training window to learn meaningful trends.

Models are compared both with raw OHLCV data and with additional technical indicators to see if these improve predictive accuracy. The evaluation includes error metrics like mean absolute error (MAE) and root mean squared error (RMSE), as well as directional accuracy, which measures how often the model correctly predicts the next day's price movement. This systematic testing allows us to select the best configuration for each stock before generating final predictions.

Although several years of historical stock data is collected for model development, the final prediction for tomorrow's price uses only the most recent portion of data defined by the selected training window. The longer history is primarily used to test different configurations. This includes how many past days to use as input (lag) and the size of the training window. These tests allow identification of the setup that performs best. Once these parameters are determined, the model only "looks back" at the recent window of data when making the actual next-day prediction, just as a trader would rely on the latest market information.

Comparing the best-performing configurations for Microsoft (MSFT) shows the impact of technical indicators:

Feature Set	Lag	Training Window	MAE	MAPE	R^2	Directional Accuracy
Without TA	5	120	5.42	1.08%	0.901	48.8%
With TA	4	100	4.88	0.97%	0.911	52.9%

Including technical indicators provides a modest but consistent improvement, with lower MAE and MAPE, slightly higher R^2, and improved directional accuracy. This shows that augmenting raw OHLCV data with carefully selected technical features can enhance predictive performance while keeping the model relatively simple.

To illustrate how optimal configurations differ between stocks, the table below summarizes the best-performing lag, training window, and resulting metrics for MAPE and directional accuracy (DA) for Microsoft (MSFT), Coca-Cola (KO), Johnson & Johnson (JNJ), Visa (V), and Netflix (NFLX). When tuning the lag and training window size, the objective was to achieve the lowest possible MAPE.

Ticker	Lag	Training Window	MAPE	DA
MSFT	4	100	0.986%	52.94%
KO	3	100	0.751%	55.37%
JNJ	2	80	0.961%	53.52%
V	2	90	1.028%	45.11%
NFLX	3	100	1.403%	57.02%

Visa's weaker results are likely due to its recent sideways price movement. When a stock lacks a clear trend and fluctuates up and down within a narrow range, the model finds it harder to identify consistent patterns.

Hyperparameter Optimization

The model's predictive performance is further improved by tuning its hyperparameters with Optuna. Optuna systematically explores many combinations of parameters, evaluating each configuration using the walk-forward validation process.

Different optimization goals can be targeted, such as minimizing MAE, MAPE, or maximizing directional accuracy, depending on whether the focus is on precise price prediction or correctly forecasting price movement direction. By combining historical data, lagged features, and technical indicators, Optuna helps identify the hyperparameter settings that provide the best balance of accuracy and robustness for each individual stock.

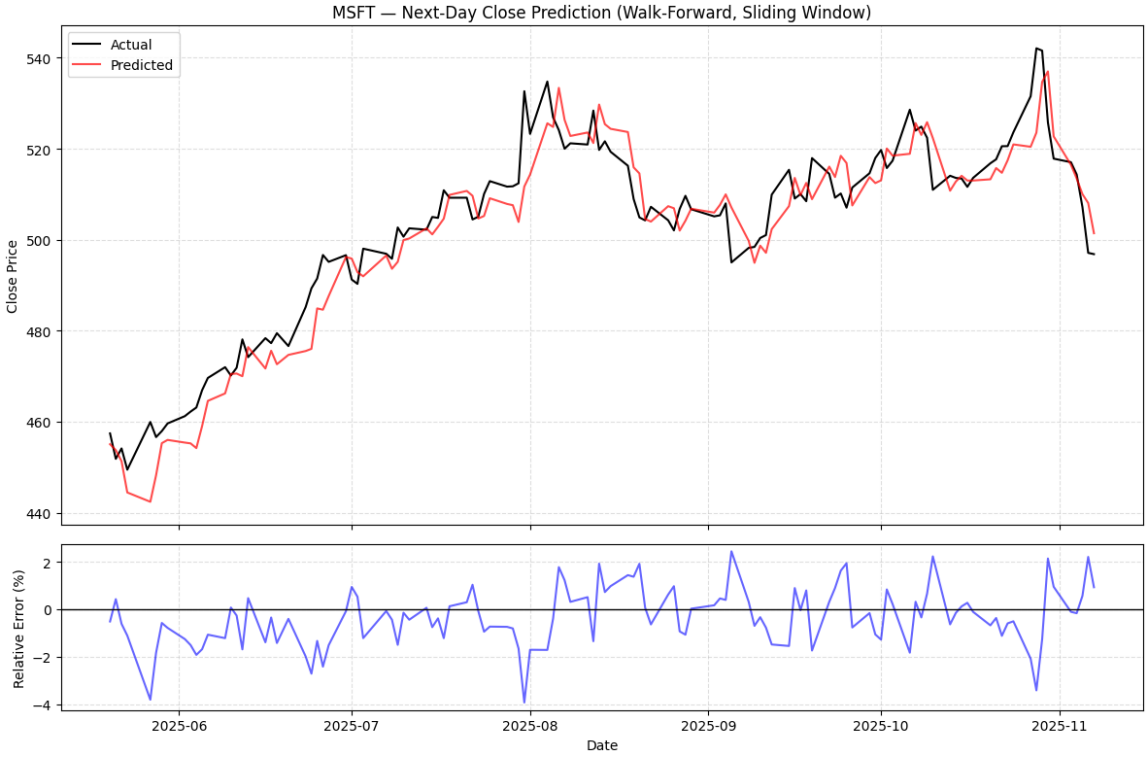
Several optimization objectives were explored: MAPE (Mean Absolute Percentage Error), a combined metric of MAPE and directional accuracy (DA), and DA alone. MAPE was primarily used because it directly measures the relative size of prediction errors in percentage terms, making it intuitive and comparable across stocks with different price levels. Optimizing for DA, on the other hand, focuses only on predicting whether the price will go up or down, without regard for the magnitude of the prediction error. This can conflict with minimizing MAPE because correctly predicting the direction does not guarantee a small percentage error. For example, a model could correctly anticipate that a stock will rise, but if it overestimates the size of the increase, the MAPE will be large. Conversely, a model could predict the next-day price very accurately in magnitude but get the direction wrong if the actual change is small.

Since this is a regression model, it is fundamentally designed to predict numeric prices, so error metrics like MAPE, MAE, or RMSE are the natural focus. Optimizing for directional accuracy (DA) within a regression framework is somewhat indirect: the model is not explicitly trained to classify "up" or "down," so improvements in DA are incidental rather than guaranteed. If the primary goal is predicting the direction of stock movement rather than exact prices, a classification model (e.g., logistic regression, XGBoost classifier, or other tree-based classifiers) would be more appropriate.

The results for different optimization objectives for Microsoft (MSFT) are shown in the following table. Optimizing for the combined MAPE + DA metric yields the lowest prediction errors, while optimizing solely for directional accuracy enhances the model's ability to correctly anticipate the direction of price movements.

Optimization Objective	Best MAPE	Best DA
MAPE	0.98607	52.94%
MAPE + DA (combined)	0.94722	52.94%
DA	0.98031	58.82%

The walk-forward plot shows Microsoft's actual versus predicted next-day closing prices over the test period. The upper panel compares predicted and true prices. It shows how closely the model follows daily movements. The lower panel displays the relative prediction error in percentage terms, providing insight into the model's day-to-day accuracy.



Conclusions

Predicting short-term stock prices is inherently challenging due to the stochastic and noisy nature of market time series. Using the developed XGBoost model, typical next-day predictions achieved a MAPE of around 1% and directional accuracy ranging from approximately 52% to 57%. When specifically optimized for directional accuracy, the model can achieve up to 60% accuracy for certain stocks, which is similar to the benchmark reported in *An Introduction to Statistical Learning: With Applications in Python* [4].

Adding technical indicators yielded a small yet consistent gain in prediction quality, slightly reducing errors and increasing the proportion of variance explained. Directional accuracy also improved, indicating that incorporating these features helps the model better capture short-term price movements. These results suggest that technical analysis can provide valuable information beyond raw price and volume data for next-day forecasting.

Although the results show that machine learning can offer useful insights beyond basic rules, the model still needs a lot of fine-tuning before it could be used in a real trading setting. Hyperparameter optimization is computationally intensive, and each stock generally requires its own tailored configuration. Overall, the model provides a solid foundation for next-day price prediction, but practical use would require further refinement and careful evaluation.

References

[1] Xu, J., He, J., Gu, J., Wu, H., Wang, L., Zhu, Y., ... & Zhou, Z. (2022). Financial time series prediction based on XGBoost and generative adversarial networks. *International Journal of Circuits, Systems and Signal Processing*, *16*, 637-645.

[2] Guennioui, O., Chiadmi, D., & Amghar, M. (2023). 'Improving global stock market prediction with XGBOOST and LightGBM machine learning models. *Review of Economics and Finance*, *21*, 2603-2610.

[3] Vuong, P. H., Dat, T. T., Mai, T. K., & Uyen, P. H. (2022). Stock-price forecasting based on XGBoost and LSTM. *Computer Systems Science & Engineering*, *40*(1).

[4] James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. https://doi.org/10.1007/978-3-031-38747-0